به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس الگوریتم پیشرفته

تمرین دستی دوم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/01/04
استاد	دکتر هشام فیلی
رشته تحصیلی	مهندسی کامپیوتر-گرایش معماری

فهرست

سوال 1 – مسئله مجموع زیر مجموعه 4

1.1 تحلیل مسئله 4

صورت مسئله: 4

1.2 روش حل مسئله: 4

روابط بازگشتی: 4

گام‌های الگوریتم: 4

1.3 پیچیدگی زمانی و فضایی: 5

1.4 مثال: 5

سوال 2 – مسئله ببرهای در قفس 6

2.1 تحلیل مسئله 6

2.2 الگوریتم برنامه‌نویسی پویا 7

2.3 مثال 8

2.4 روش بازیابی تخصیص بهینه 10

2.5 پیچیدگی زمانی 12

سوال 3 – خرید و فروش نفت 14

3.1 تحلیل مسئله 14

3.2 گام‌های الگوریتم 15

3.3 مثال 16

3.4 تحلیل زمانی الگوریتم 18

سوال 4 – اتصال طناب ها 19

4.1 تحلیل مسئله 19

تعریف دقیق مسئله: 19

4.2 راه حل: 19

4.3 اثبات درستی الگوریتم حریصانه 20

4.4 تحلیل پیچیدگی: 20

سوال 5 – توزیع جایزه 22
5.1 تحلیل مسئله 22
5.2 توضیح الگوریتم:..... 22
5.3 اثبات درستی: 22
5.4 مثال عددی: 23
5.5 تحلیل پیچیدگی:..... 25
سوال 6 – حداقل زمان برای اتمام کارها..... 26
6.1 تحلیل مسئله 26
6.2 الگوریتم پیشنهادی..... 27
6.3 مثال عددی 28
6.4 تحلیل زمانی الگوریتم..... 29
6.5 اثبات درستی الگوریتم..... 29

شکل‌ها

شکل 1- کد الگوریتم Maximum Sum of Non-Adjacent Elements در پایتون 6
شکل 2 - کد تخصیص قفس به ببرها با برنامه نویسی پویا..... 13
شکل 3- کد بازیابی تخصیص بهینه..... 13
شکل 4- کد الگوریتم گریدی برای مسئله اتصال طناب ها..... 21
شکل 5- کد الگوریتم حریصانه برای اختصاص کتاب 26
شکل 6- کد الگوریتم سوال 6 مربوط به workload 30

سوال 1 – مسئله مجموع زیر مجموعه

1.1 تحلیل مسئله

صورت مسئله:

سوال ۱: مسئله مجموع زیرمجموعه

یک دنباله شامل n عدد (مثبت یا منفی) به شما داده شده است. وظیفه‌ی شما انتخاب یک زیرمجموعه از این اعداد است به‌طوری که مجموع آن حداکثر شود، با این محدودیت که نمی‌توانید دو عدد متوالی را انتخاب کنید. یعنی اگر x_i را انتخاب کنید، نباید x_{i-1} یا x_{i+1} را انتخاب کنید.

توضیح دهید که چگونه می‌توان در زمان چندجمله‌ای نسبت به n، این زیرمجموعه را به‌گونه‌ای یافت که مجموع کل آن حداکثر شود.

این مسئله یک نوع از مسئله انتخاب زیرمجموعه بهینه است که در آن باید زیرمجموعه‌ای از اعداد را از یک دنباله انتخاب کنیم، به‌طوری‌که مجموع اعداد زیرمجموعه بیشینه شود. محدودیت مسئله این است که در انتخاب هر عنصر، نمی‌توان دو عنصر متوالی را انتخاب کرد.

این مسئله با استفاده از **برنامه‌ریزی پویا (Dynamic Programming)** قابل حل است.

1.2 روش حل مسئله:

برای حل مسئله، از یک آرایه‌ی کمکی به نام dp استفاده می‌کنیم که در آن:

- dp[i] نشان‌دهنده‌ی بیشینه مجموع زیرمجموعه‌ای است که شامل اعداد تا اندیس i باشد.

روابط بازگشتی:

اگر عنصر i را انتخاب کنیم، نمی‌توانیم عنصر i-1 را انتخاب کنیم. در این حالت، مجموع برابر خواهد بود با:

$$dp[i] = arr[i] + dp[i-2]$$

اگر عنصر i را انتخاب نکنیم، مجموع برابر خواهد بود با:

$$dp[i] = dp[i-1]$$

بنابراین رابطه بازگشتی به صورت زیر است:

$$dp[i] = \max\left(dp[i-1], arr[i] + dp[i-2]\right)$$

گام‌های الگوریتم:

1. ابتدا یک آرایه dp به طول n ایجاد می‌کنیم.

2. مقادیر اولیه:

اگر فقط یک عنصر داشته باشیم، همان عنصر انتخاب می‌شود

$$dp[0] = arr[0]$$

برای دو عنصر، بزرگ‌تر را انتخاب می‌کنیم

$$dp[1] = \max\,(arr[0], arr[1])$$

3. از اندیس i = 2 تا i = n-1، مقدار dp[i] را با استفاده از رابطه بازگشتی محاسبه می‌کنیم.
4. مقدار نهایی در dp[n-1]، بیشینه مجموع زیرمجموعه خواهد بود.

1.3 پیچیدگی زمانی و فضایی:

پیچیدگی زمانی: $O(n)$ ، زیرا تنها یک بار از ابتدای آرایه تا انتها پیمایش می‌کنیم.

پیچیدگی فضایی: $O(n)$، به دلیل استفاده از آرایه dp. اما می‌توان آن را به $O(1)$ کاهش داد، زیرا برای محاسبه‌ی $dp[i]$ تنها به دو مقدار قبلی (dp[i-1] و dp[i-2]) نیاز داریم.

1.4 مثال:

فرض کنید دنباله‌ی زیر داده شده باشد:

$$arr = [3,2,7,10]$$

مراحل اجرا:

محاسبه مقادیر اولیه:

$$dp[0] = 3$$

$$dp[1] = \max(3,2) = 3$$

محاسبات برای i = 2 :

$$dp[2] = \max(dp[1], arr[2] + dp[0]) = \max(3,7+3) = 10$$

محاسبات برای i = 3 :

$$dp[3] = \max(dp[2], arr[3] + dp[1]) = \max(10,10+3) = 13$$

بیشینه مجموع زیرمجموعه برابر است با $dp[3] = 13$

جوابی که در بالا در آوردیم برای مقدار پاسخ بهینه است و برای خود جواب باید از یک آرایه دیگر استفاده کنیم و به شکل برگشتی اندیس خانه هایی از آرایه را که انتخاب کردیم ذخیره کنیم.

جمع بندی: این مسئله با نام Maximum Sum of Non-Adjacent Elements هم شناخته میشود.

در صفحه بعد کد مربوط به این الگوریتم رو می‌تونید به همراه خروجی اش مشاهده کنید.

```python
def maxSubsetSum(arr):
    n = len(arr)
    if n == 0:
        return 0
    if n == 1:
        return arr[0]

    dp = [0] * n
    dp[0] = arr[0]
    dp[1] = max(arr[0], arr[1])

    for i in range(2, n):
        dp[i] = max(dp[i-1], arr[i] + dp[i-2])

    return dp[-1]

# Example
arr = [3, 2, 7, 10]
print(maxSubsetSum(arr))  # Output: 13
```

```
PS C:\Users\Asus> & C:/Users/Asus/AppData/Local/Programs/Python/Python312/python.exe "d:/Advance Algorithm/codes/Algorithm.py"
13
PS C:\Users\Asus>
```

شکل 1- کد الگوریتم **Maximum Sum of Non-Adjacent Elements** در پایتون

سوال 2 – مسئله ببرهای در قفس

2.1 تحلیل مسئله

سوال ۲: ببرهای در قفس

فرهاد شخصیتی عجیب است که n ببر خانگی و n^2 قفس دارد. هر ببر i دارای سن مثبت a_i و اندازه s_i است (هیچ دو ببری دارای سن یا اندازه‌ی یکسان نیستند). هر قفس j دارای ظرفیت مثبت c_j و فاصله‌ی مشخص d_j از اتاق خواب فرهاد است (هیچ دو قفسی دارای ظرفیت یا فاصله‌ی یکسان نیستند). فرهاد باید هر ببر را در یک قفس مجزا قرار دهد.

شرایط قرارگیری ببرها در قفس‌ها:

- فرهاد ببرهای مسن‌تر را بیشتر دوست دارد و می‌خواهد آن‌ها را نزدیک‌تر به اتاق خواب خود قرار دهد. یعنی اگر سن یک ببر x کمتر از سن یک ببر y باشد، آنگاه ببر x باید در قفسی قرار گیرد که از قفس ببر y دورتر باشد.
- اگر ببر i در قفس c_j قرار داده شود، اگر $s_i > c_j$ باشد ناراحتی آن برابر خواهد بود با $s_i - c_j$. در غیر این صورت ببر ناراحتی نخواهد داشت.

الگوریتمی با زمان اجرای $O(n^3)$ ارائه دهید که ببرها را در قفس‌ها به‌گونه‌ای جای دهد که شرط بالا رعایت شود و مجموع ناراحتی ببرها حداقل گردد.

در این مسئله باید **n** ببر را در **n^2** قفس قرار دهیم به طوری که:

1. هر ببر **i** دارای سن a_i و اندازه s_i است.
2. هر قفس **j** دارای ظرفیت **cj** و فاصله **dj** از اتاق خواب است.
3. **شرط اول**: ببرهای مسن‌تر باید نزدیک‌تر به اتاق خواب قرار گیرند (یعنی اگر سن ببر x کمتر از سن ببر y باشد، ببر x باید در قفسی قرار گیرد که از قفس ببر y دورتر باشد).
4. **شرط دوم**: اگر اندازه ببر بزرگتر از ظرفیت قفس باشد، ناراحتی برابر با تفاضل آنها است $(s_i - c_j)$. در غیر این صورت، ناراحتی نخواهد داشت.
5. هدف: حداقل کردن مجموع ناراحتی ببرها.

رویکرد برنامه‌نویسی پویا

برای حل این مسئله، از رویکرد **زیرساخت بهینه (Optimal Substructure)** استفاده می‌کنیم:

تعریف تابع DP:

DP[i][j]=حداقل ناراحتی ممکن برای قرار دادن ببرهای 1 تا i در j قفس اول (با توجه به ترتیب فاصله)

2.2 الگوریتم برنامه‌نویسی پویا

پیش‌پردازش:

1. ببرها را بر اساس سن به صورت نزولی مرتب می‌کنیم.
2. قفس‌ها را بر اساس فاصله از اتاق خواب به صورت صعودی مرتب می‌کنیم.

تعریف تابع بازگشتی:

برای حل این مسئله، تابع DP[i][j] را به صورت زیر تعریف می‌کنیم:

- DP[i][j] = حداقل ناراحتی برای تخصیص i ببر اول به j قفس اول

DP[i][j] = min (DP[i-1] [j-1] + discomfort (i, j), DP[i][j-1])

که در آن:

Discomfort (i, j) = ناراحتی ببر i در قفس j:

اگر $s_i > c_j$ باشد: $s_i - c_j$

در غیر این صورت: 0

2.3 مثال

برای درک بهتر، یک مثال ساده را بررسی می‌کنیم:

ورودی:

n = 3 (تعداد ببرها)

سن ببرها (a): [10, 15, 5]

اندازه ببرها (s): [8, 12, 6]

ظرفیت قفس‌ها (c): [7, 10, 5, 8, 9, 11, 6, 7, 8]

فاصله قفس‌ها (d): [3, 5, 8, 2, 6, 9, 4, 7, 1]

نکته مهم) دقت شود که اندیس آرایه ظرفیت قفس ها و فاصله آنها شماره قفس در ابتدای کار را نشان میدهد و از 0 تا 8 می‌باشد.

مراحل حل:

1. مرتب‌سازی:

مرتب‌سازی ببرها (بر اساس سن نزولی):

```
tigers = [(i, a[i], s[i]) for i in range(n)]
```

در این مرتب‌سازی:

- اندیس اول (i): شماره اصلی ببر در ورودی
- اندیس دوم (a[i]): سن ببر
- اندیس سوم (s[i]): اندازه ببر

بعد از مرتب‌سازی:

```
Tigers = [(1, 15, 12), (0, 10, 8), (2, 5, 6)]
```

مرتب‌سازی قفس‌ها (بر اساس فاصله صعودی):

```
cages = [(j, d[j], c[j]) for j in range(n*n)]
```

در این مرتب‌سازی:

- اندیس اول (j): شماره اصلی قفس در ورودی
- اندیس دوم (d[j]): فاصله قفس از اتاق خواب
- اندیس سوم (c[j]): ظرفیت قفس

بعد از مرتب‌سازی:

Cages = [(8, 1, 8), (3, 2, 8), (0, 3, 7), (6, 4, 6), (1, 5, 10), (4, 6, 9), (7, 7, 7), (2, 8, 5), (5, 9, 11)]

2. محاسبه جدول DP:

ابتدا جدول DP را با مقادیر بی‌نهایت پر می‌کنیم و DP[0][0] = 0 قرار می‌دهیم.

برای هر ببر i و هر قفس j، دو حالت داریم:

1. ببر i در قفس j قرار نگیرد: DP[i][j] = DP[i][j-1]
2. ببر i در قفس j قرار گیرد: DP[i][j] = DP[i-1][j-1] + discomfort(i, j)

برای مثال، محاسبه DP[1][1]:

- ببر 1 (با سن 15 و اندازه 12) و قفس 1 (با فاصله 1 و ظرفیت 8)
- ناراحتی = max(0, 12 - 8) = 4
- DP [1][1] = min (DP [0][0] + 4, DP [1][0]) = min (0 + 4, ∞) = 4

به همین ترتیب برای DP[2][2]:

- ببر 2 (با سن 10 و اندازه 8) و قفس 2 (با فاصله 2 و ظرفیت 8)
- ناراحتی = max(0, 8 - 8) = 0
- DP [2][2] = min (DP [1][1] + 0, DP [2][1]) = min (4 + 0, 4) = 4

و برای DP[3][3]:

- ببر 3 (با سن 5 و اندازه 6) و قفس 3 (با فاصله 3 و ظرفیت 7)
- ناراحتی = max(0, 6 - 7) = 0
- DP [3][3] = min (DP [2][2] + 0, DP [3][2]) = min (4 + 0, 4) = 4

جدول نهایی DP (فقط بخشی از آن):

	0	1	2	3
0	0	∞	∞	∞
1	∞	4	4	4
2	∞	∞	4	4
3	∞	∞	∞	4

پس از تکمیل جدولDP ، باید تخصیص بهینه ببرها به قفس‌ها را بازیابی کنیم. این بخش یکی از مهم‌ترین قسمت‌های الگوریتم برنامه‌نویسی پویا است که نشان می‌دهد دقیقاً کدام ببر در کدام قفس قرار می‌گیرد.

2.4 روش بازیابی تخصیص بهینه

گام‌های بازیابی:

1. از سلول نهایی جدول DP یعنی DP[n][n] شروع می‌کنیم.
2. در هر مرحله، باید تصمیم بگیریم که آیا ببر فعلی در قفس فعلی قرار می‌گیرد یا نه.
3. با حرکت به عقب در جدول، تخصیص کامل را بازیابی می‌کنیم.

بازیابی گام به گام:

شروع از DP[3][3] = 4:

مقایسه با DP[3][2] = ∞:

چون $DP[3][3] \neq DP[3][2]$ ، ببر 3 در قفس 3 قرار می‌گیرد.

ببر با شماره اصلی 2 (سومین ببر در لیست مرتب شده) به قفس با شماره اصلی 0 تخصیص می‌یابد.

Allocation [2] = 0

به $DP[2][2]$ می‌رویم.

بررسی DP[2][2] = 4:

مقایسه با DP[2][1] = 4:

چون $DP[2][2] = DP[2][1]$ ، ببر 2 در قفس 2 قرار نمی‌گیرد.

به $DP[2][1]$ می‌رویم.

بررسی DP[2][1] = 4:

مقایسه با DP[2][0] = ∞:

چون $DP[2][1] \neq DP[2][0]$، ببر 2 در قفس 1 قرار می‌گیرد.

ببر با شماره اصلی 0 (دومین ببر در لیست مرتب شده) به قفس با شماره اصلی 3 تخصیص می‌یابد.

allocation [0] = 3

- به $DP[1][0]$ می‌رویم.

بررسی $DP[1][0] = \infty$:

این حالت نامعتبر است (نمی‌توانیم ببر داشته باشیم ولی قفسی نداشته باشیم).

به حالت دیگری می‌رویم و بررسی می‌کنیم که آیا DP[1][1] = 4 از DP[1][0] + discomfort آمده است یا نه.

ببر با شماره اصلی 1 (اولین ببر در لیست مرتب شده) به قفس با شماره اصلی 8 تخصیص می‌یابد.

Allocation [1] = 8

نتیجه نهایی:

- allocation = [3, 8, 0]
- یعنی:
  - ببر 0 (سن: 10، اندازه: 8) در قفس 3 (فاصله: 2، ظرفیت: 8)
  - ببر 1 (سن: 15، اندازه: 12) در قفس 8 (فاصله: 1، ظرفیت: 8)
  - ببر 2 (سن: 5، اندازه: 6) در قفس 0 (فاصله: 3، ظرفیت: 7)

چرا این روش بازیابی درست است؟

در برنامه‌نویسی پویا، جدول DP حاوی مقادیر بهینه برای زیرمسائل است، اما تصمیم‌هایی که به این مقادیر بهینه منجر شده‌اند را ذخیره نمی‌کند. بنابراین، برای بازیابی تخصیص بهینه، باید با بررسی مقادیر جدول DP، تصمیم‌هایی که گرفته شده را بازسازی کنیم.

در هر مرحله، دو حالت وجود دارد:

1. **ببر فعلی در قفس فعلی قرار نمی‌گیرد**: در این صورت، DP[i][j] = DP[i][j-1]
2. **ببر فعلی در قفس فعلی قرار می‌گیرد**: در این صورت، DP[i][j] = DP[i-1][j-1] + discomfort(i, j)

با مقایسه مقادیر DP[i][j] و DP[i][j-1] می‌توانیم تشخیص دهیم که کدام حالت رخ داده است.

نکات مهم در بازیابی تخصیص:

1. **حفظ شماره‌های اصلی**: در هنگام مرتب‌سازی ببرها و قفس‌ها، شماره اصلی آن‌ها را نگه می‌داریم تا در مرحله بازیابی بتوانیم تخصیص صحیح را انجام دهیم.

2. **شروع از انتها**: بازیابی تخصیص را از آخرین سلول جدول DP شروع می‌کنیم و به سمت عقب حرکت می‌کنیم.
3. **بررسی شرایط مرزی**: در حین بازیابی، باید مراقب شرایط مرزی (مانند i=0 یا j=0) باشیم.
4. **اطمینان از صحت تخصیص**: پس از بازیابی، باید بررسی کنیم که تخصیص به‌دست‌آمده تمام شرایط مسئله را برآورده می‌کند:
   - هر ببر در یک قفس قرار گرفته است.
   - ببرهای مسن‌تر در قفس‌های نزدیک‌تر به اتاق خواب قرار گرفته‌اند.
   - مجموع ناراحتی حداقل است.

با این روش بازیابی، می‌توانیم تخصیص بهینه ببرها به قفس‌ها را با دقت و کارایی بازسازی کنیم.

2.5 پیچیدگی زمانی

1. **مرتب‌سازی ببرها**: $O(n \log n)$
2. **مرتب‌سازی قفس‌ها**: $O(n^2 \log n^2) = O(n^2 \log n)$
3. **محاسبه جدول DP**: $O(n^2)$
4. **بازیابی تخصیص**: $O(n)$

بنابراین، پیچیدگی زمانی کلی الگوریتم برابر است با: $\mathbf{O(n^2 \log n + n^2) = O(n^2 \log n)}$

این پیچیدگی در محدوده خواسته‌شده $\mathbf{O(n^3)}$ قرار دارد.

اما اگر بخواهیم دقیق‌تر بررسی کنیم، پیچیدگی اصلی در محاسبه جدول DP است:

- برای هر ببر (n)، باید برای هر قفس (n) محاسبه انجام دهیم.
- برای هر محاسبه، باید قفس‌های قبلی را بررسی کنیم (در بدترین حالت n^2).

بنابراین پیچیدگی زمانی دقیق‌تر: $\mathbf{O(n^3)}$ است.

جمع بندی:

کد این الگوریتم:

در صفحه بعد میتونید کدهای مربوط به خود الگوریتم و تخصیص بازیابی رو مشاهده کنید.

```
D: > Advance Algorithm > codes > algorithmcode.py > ...
function tigerAllocationDP(n, a, s, c, d):
    # مرتب‌سازی ببرها بر اساس سن (نزولی)
    tigers = [(i, a[i], s[i]) for i in range(n)]
    tigers.sort(key=lambda x: x[1], reverse=True)

    # مرتب‌سازی قفس‌ها بر اساس فاصله (صعودی)
    cages = [(j, d[j], c[j]) for j in range(n*n)]
    cages.sort(key=lambda x: x[1])

    # مقداردهی اولیه DP
    DP = [[float('inf')] * (n+1) for _ in range(n+1)]
    DP[0][0] = 0

    # محاسبه تابع DP
    for i in range(1, n+1):
        for j in range(1, n+1):
            # قرار نگیرد j در قفس i حالت 1: ببر
            if j > 0:
                DP[i][j] = DP[i][j-1]

            # قرار گیرد j در قفس i حالت 2: ببر
            tiger_idx, tiger_age, tiger_size = tigers[i-1]
            cage_idx, cage_distance, cage_capacity = cages[j-1]

            discomfort = max(0, tiger_size - cage_capacity)
            if i > 0 and j > 0:
                DP[i][j] = min(DP[i][j], DP[i-1][j-1] + discomfort)
```

شکل 2 - کد تخصیص قفس به ببرها با برنامه نویسی پویا

```
    allocation = [-1] * n  # آرایه برای ذخیره تخصیص نهایی
    i = n  # شروع از آخرین ببر
    j = n  # شروع از آخرین قفس

    while i > 0 and j > 0:
        # قرار می‌گیرد یا نه j در قفس i بررسی اینکه آیا ببر
        if DP[i][j] == DP[i][j-1]:
            # قرار نمی‌گیرد، به قفس قبلی می‌رویم j در قفس i ببر
            j = j - 1
        else:
            # قرار می‌گیرد j در قفس i ببر
            tiger_idx = tigers[i-1][0]  # شماره اصلی ببر
            cage_idx = cages[j-1][0]    # شماره اصلی قفس

            allocation[tiger_idx] = cage_idx  # تخصیص ببر به قفس

            # به ببر قبلی و قفس قبلی می‌رویم
            i = i - 1
            j = j - 1

    return allocation, DP[n][n]
```

شکل 3- کد بازیابی تخصیص بهینه

سوال 3 – خرید و فروش نفت

3.1 تحلیل مسئله

سوال 3: خرید و فروش نفت

ایران این ماه m بشکه نفت تولید خواهد کرد. فهرستی شامل n سفارش از خریداران بالقوه داریم. در سفارش i، خریدار مایل است a_i بشکه را با قیمت کل p_i خریداری کند. (قیمت کل، نه قیمت هر بشکه؛ همچنین مقدار p_i ممکن است منفی باشد)

شرایط فروش:

۳

- هر سفارش یا باید به‌طور کامل پذیرفته شود یا به‌طور کامل رد شود. (نمی‌توان تنها بخشی از سفارش را تأمین کرد)
- هر سفارش فقط یک بار می‌تواند پذیرفته شود.
- مجبور نیستیم تمام نفت خود را بفروشیم، اما برای هر بشکه‌ی فروخته‌نشده باید هزینه‌ی ذخیره‌سازی به مقدار s تومان بپردازیم.

الگوریتمی با زمان اجرای $O(nm)$ طراحی کنید که تعیین کند کدام سفارش‌ها باید پذیرفته شوند تا سود حداکثر شود. (توجه کنید که سود می‌تواند منفی باشد)

صورت مسئله

ایران این ماه **m** بشکه نفت تولید خواهد کرد. فهرستی شامل **n** سفارش از خریداران بالقوه داریم. در سفارش **i**، خریدار مایل است **ai** بشکه را با قیمت کل **pi** خریداری کند (قیمت کل، نه قیمت هر بشکه؛ همچنین مقدار **pi** ممکن است منفی باشد).

شرایط فروش:

- هر سفارش یا باید به‌طور کامل پذیرفته شود یا به‌طور کامل رد شود
- هر سفارش فقط یک بار می‌تواند پذیرفته شود
- مجبور نیستیم تمام نفت خود را بفروشیم، اما برای هر بشکه‌ی فروخته‌نشده باید هزینه‌ی ذخیره‌سازی به مقدار **s** تومان بپردازیم
- باید الگوریتمی با زمان اجرای **O(nm)** طراحی کنیم

راه حل ساده و گام به گام

ما می‌خواهیم تصمیم بگیریم کدام سفارش‌ها را بپذیریم تا سود حداکثر شود. برای این کار از برنامه‌نویسی پویا استفاده می‌کنیم.

تعریف جدول DP

DP[i][j]= حداکثر سود ممکن با بررسی i سفارش اول و با داشتن j بشکه نفت باقی‌مانده (از کل mبشکه)

3.2 گام‌های الگوریتم

گام 1: مقداردهی اولیه

برای حالتی که هیچ سفارشی نداریم (i = 0)، سود برابر با منفی هزینه ذخیره‌سازی است:

- DP[0][j] = -s × j برای همه j از 0 تا m

گام 2: پر کردن جدول DP

حالا جدول DP را از بالا به پایین و از چپ به راست پر می‌کنیم:

برای هر سفارش i (از 1 تا n) و برای هر مقدار j بشکه باقی‌مانده (از 0 تا m):

اگر ai ≤ j (یعنی می‌توانیم سفارش را قبول کنیم)، دو گزینه داریم:

قبول سفارش: DP[i-1][j-ai] + pi

رد سفارش: DP[i-1][j]

بهترین گزینه را انتخاب می‌کنیم: DP[i][j] = max(DP[i-1][j-ai] + pi, DP[i-1][j])

اگر j < ai (نمی‌توانیم سفارش را قبول کنیم):

- DP[i][j] = DP[i-1] [j]

گام 3: یافتن پاسخ نهایی

پاسخ نهایی در DP[n][m] خواهد بود.

گام 4: یافتن سفارش‌های پذیرفته شده

حالا می‌خواهیم بدانیم کدام سفارش‌ها را باید بپذیریم. برای این کار، از انتهای جدول شروع می‌کنیم و به عقب برمی‌گردیم.

3.3 مثال

حال با بررسی یک مثال تمامی مرحل این الگوریتم ارائه داده شده را بررسی می‌کنیم:

فرض کنیم:

- m = 10 (تعداد بشکه‌های نفت)
- n = 3 (تعداد سفارش‌ها)
- سفارش‌ها: [(a_1=3, p_1=20), (a_2=5, p_2=30), (a_3=4, p_3=25)]
- s = 2 (هزینه ذخیره‌سازی هر بشکه)

گام 1: مقداردهی اولیه

DP [0][0] = 0

DP [0][1] = -2

DP [0][2] = -4

...

DP [0][10] = -20

این مقادیر نشان می‌دهد اگر هیچ سفارشی را نپذیریم، چقدر ضرر می‌کنیم.

برای i=1 (سفارش اول):

- DP [1][0] = DP [0][0] = 0 (رد کردن)
- DP [1][1] = DP [0][1] = -2 (رد کردن)

- DP [1][2] = DP [0][2] = -4 (رد کردن)
- DP [1][3] = max (DP [0][3], DP [0][0] + 20) = max (-6, 20) = 20 (پذیرفتن)
- DP [1][4] = max (DP [0][4], DP [0][1] + 20) = max (-8, 18) = 18 (پذیرفتن)
- ...
- DP [1][10] = max (DP [0][10], DP [0][7] + 20) = max (-20, -14 + 20) = 6 (پذیرفتن)

برای i=2 (سفارش دوم):

- DP [2][0] = DP [1][0] = 0 (رد کردن)
- ...
- DP [2][5] = max (DP [1][5], DP [1][0] + 30) = max (16, 30) = 30 (پذیرفتن)
- ...
- DP [2][10] = max (DP [1][10], DP [1][5] + 30) = max (6, 16 + 30) = 46 (پذیرفتن)

برای i=3 (سفارش سوم):

...

DP [3][10] = max (DP [2][10], DP [2][6] + 25) = max (46, 32 + 25) = 57 (پذیرفتن)

گام 3: پاسخ نهایی

پاسخ نهایی DP[3][10] = 57 است، یعنی حداکثر سود ممکن 57 واحد است.

گام 4: یافتن سفارش‌های پذیرفته شده

برای تعیین اینکه کدام سفارش‌ها پذیرفته شده‌اند، از آخر به اول حرکت می‌کنیم:

1. از $DP[3][10]$ شروع می‌کنیم
2. اگر $DP[3][10] \neq DP[2][10]$ باشد، یعنی سفارش 3 پذیرفته شده است
3. سپس به $DP[2][10-4] = DP[2][6]$ می‌رویم (چون 4 بشکه برای سفارش 3 استفاده کردیم)
4. و به همین ترتیب ادامه می‌دهیم

در این مثال، سفارش‌های 2 و 3 پذیرفته می‌شوند (5+4=9 بشکه) و 1 بشکه باقی می‌ماند که هزینه ذخیره‌سازی آن 2 واحد است. سود کل $30 + 25 - 2 = 53$ واحد.

3.4 تحلیل زمانی الگوریتم

پیچیدگی زمانی

الگوریتم ما دارای پیچیدگی زمانی O(nm) است که دقیقاً مطابق با خواسته مسئله است. این پیچیدگی از اجزای زیر تشکیل شده است:

1. **مقداردهی اولیه جدول DP:**

یک حلقه با O(m) اجرا برای مقداردهی DP[0][j] برای همه j از 0 تا m

2. **پر کردن جدول DP:**

دو حلقه تودرتو که حلقه بیرونی روی سفارش‌ها (n) و حلقه داخلی روی تعداد بشکه‌های باقی‌مانده (m) می‌چرخد

تعداد کل عملیات: n × m

هر عملیت شامل یک مقایسه و چند عمل محاسباتی ساده است که همگی O(1) هستند

3. **بازیابی سفارش‌های پذیرفته شده:**

یک حلقه با حداکثر n تکرار که در هر تکرار عملیات ساده O(1) انجام می‌دهد

پیچیدگی این بخش: O(n)

دلیل بهینه بودن O(nm)

1. **لزوم بررسی تمام سفارش‌ها:**

ما باید برای هر سفارش تصمیم بگیریم که آن را بپذیریم یا رد کنیم، بنابراین حداقل باید n سفارش را بررسی کنیم

2. **لزوم بررسی تمام حالت‌های ممکن برای بشکه‌های باقی‌مانده:**

برای هر سفارش، باید تمام حالت‌های ممکن برای تعداد بشکه‌های باقی‌مانده (از 0 تا m) را در نظر بگیریم

این باعث می‌شود که نیاز به بررسی m+1 حالت برای هر سفارش داشته باشیم

3. **عدم امکان کاهش پیچیدگی:**

با توجه به ماهیت مسئله که نیاز به بررسی تمام ترکیب‌های ممکن سفارش‌ها دارد، نمی‌توان پیچیدگی را از O(nm) کمتر کرد

این مسئله شبیه به مسئله کوله‌پشتی است که پیچیدگی بهینه آن نیز O(nm) است

4. **عدم وجود ساختار خاص برای کاهش پیچیدگی:**

در این مسئله هیچ ساختار خاصی (مثل مرتب‌سازی سفارش‌ها بر اساس ارزش واحد) وجود ندارد که بتواند پیچیدگی را کاهش دهد

همچنین قیمت‌ها می‌توانند منفی باشند که حل مسئله را پیچیده‌تر می‌کند

بنابراین، الگوریتم ارائه شده با پیچیدگی زمانی O(nm) بهینه است و دقیقاً با خواسته مسئله مطابقت دارد.

سوال 4 – اتصال طناب ها

4.1 تحلیل مسئله

سوال 4: اتصال طناب‌ها

آرایه‌ای شامل طول‌های چندین طناب به شما داده شده است. وظیفه‌ی شما این است که تمام طناب‌ها را به یک طناب واحد تبدیل کنید به‌گونه‌ای که مجموع هزینه‌های اتصال حداقل شود. هزینه‌ی اتصال دو طناب برابر مجموع طول‌های آن‌ها است. الگوریتمی ارائه دهید که این هزینه را حداقل کند.

تعریف دقیق مسئله:

ورودی: آرایه A شامل n طول طناب: A[1…n]

خروجی: حداقل هزینه کل برای اتصال همه طناب‌ها

هزینه اتصال دو طناب: مجموع طول آن‌ها

برای حل این مسئله از یک الگوریتم Greedy استفاده می‌کنیم.

رویکرد حریصانه: الگوریتم در هر مرحله:

1. دو طناب با کمترین طول را انتخاب می‌کند
2. آن‌ها را به هم متصل می‌کند
3. طناب جدید را به مجموعه طناب‌ها اضافه می‌کند
4. این روند تا زمانی که فقط یک طناب باقی بماند ادامه می‌یابد

4.2 راه حل:

برای حداقل کردن هزینه کل،

بهترین استراتژی استفاده از صف اولویت (Min Heap) است، چون:

همیشه باید کوتاه‌ترین طناب‌ها را اول ترکیب کنیم

پس از هر ترکیب، طناب جدید باید در جای مناسب خود قرار گیرد

این فرآیند تا زمانی که فقط یک طناب باقی بماند ادامه می‌یابد

4.3 اثبات درستی الگوریتم حریصانه: برای اثبات بهینه بودن راه حل حریصانه، از روش تناقض استفاده می‌کنیم:

فرض کنید آرایه‌ای با طول‌های [2, 3, 4, 6] داریم:

راه حل حریصانه: همیشه دو طناب کوتاه‌تر را انتخاب می‌کند

گام 1: 2+3=5 (هزینه: 5)

گام 2: 4+5=9 (هزینه: 5+9=14)

گام 3: 6+9=15 (هزینه: 14+15=29)

فرض کنیم راه حل دیگری بهینه‌تر باشد، مثلاً شروع با دو طناب بزرگتر:

گام 1: 4+6=10 (هزینه: 10)

گام 2: 2+3=5 (هزینه: 10+5=15)

گام 3: 5+10=15 (هزینه: 15+15=30)

این مثال نشان می‌دهد که شروع با طناب‌های بزرگتر منجر به هزینه بیشتری می‌شود.

چرا این الگوریتم حریصانه کار می‌کند؟

در هر مرحله، انتخاب دو طناب کوتاه‌تر بهترین انتخاب است زیرا:

طناب‌های کوتاه‌تر باید زودتر متصل شوند

تأخیر در اتصال طناب‌های بلندتر هزینه کل را کاهش می‌دهد

هر اتصال در مراحل بعدی مجدداً شمارش می‌شود

این الگوریتم با استفاده از ساختار داده heap و رویکرد حریصانه، بهینه‌ترین راه‌حل را برای مسئله اتصال طناب‌ها ارائه می‌دهد.

4.4 تحلیل پیچیدگی:

زمانی:

ساخت heap: O(n)

n-1 مرحله اتصال

هر مرحله شامل دو pop و یک push: O(log n)

پیچیدگی کل: O(n log n)

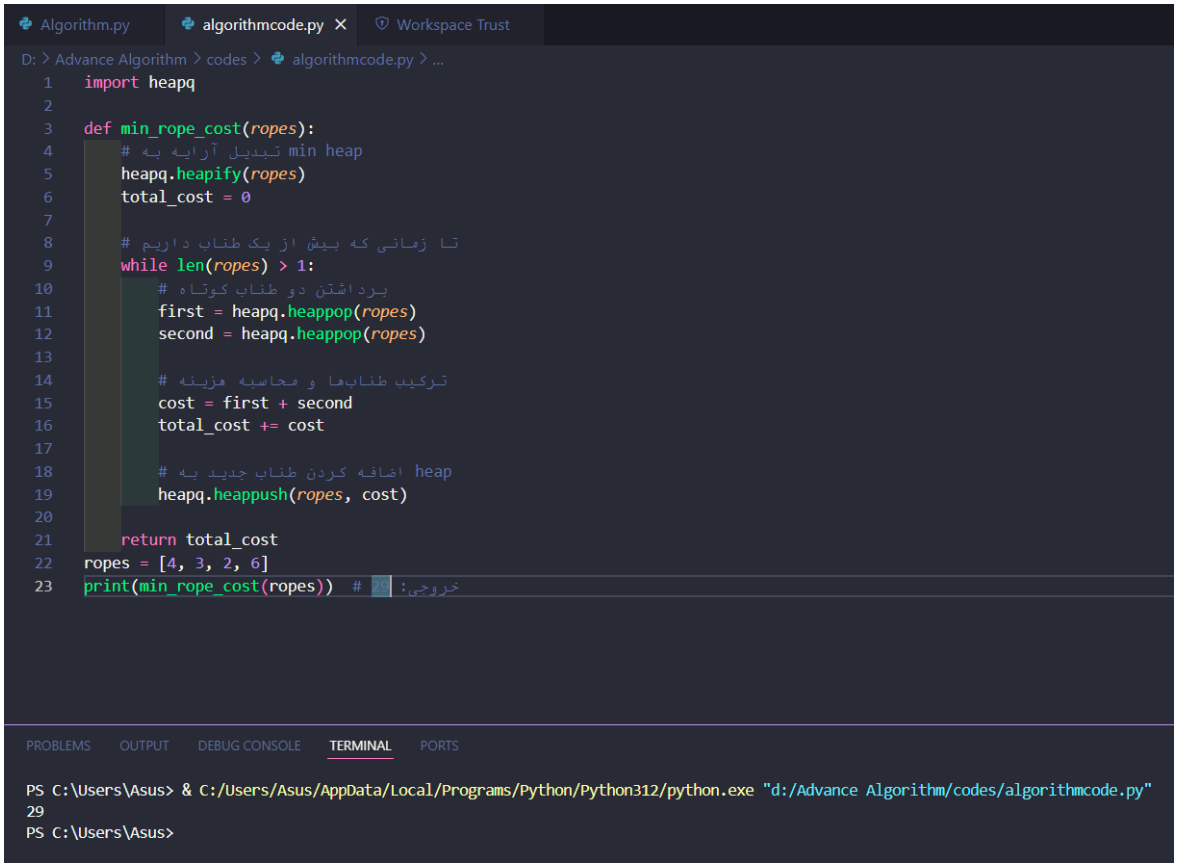
فضایی:

نیاز به یک صف اولویت: O(n)

برای اثبات بهینگی در مرحله قبل از خاصیت زیر استفاده کردیم:

- اگر $x_1 \le x_2$ و $y_1 \le y_2$، آنگاه: $x_1 + y_1 + (x_1 + y_1) \le x_2 + y_2 + (x_2 + y_2)$
- این نشان می‌دهد که اتصال طناب‌های کوتاه‌تر همیشه هزینه کمتری دارد

در نهایت کد مربوط به این الگوریتم در زبان برنامه نویسی پایتون نوشته شده و تصویر آن به همراه خروجی برای مثال آورده شده است:



شکل 4- کد الگوریتم گریدی برای مسئله اتصال طناب ها

سوال 5 – توزیع جایزه

5.1 تحلیل مسئله

سوال 5: توزیع جایزه

در یک مسابقه‌ی علمی، n شرکت‌کننده در یک صف ایستاده‌اند و پس از انجام آزمون، به هر شرکت‌کننده یک امتیاز داده شده است. مسئولین مسابقه تصمیم گرفته‌اند به هر شرکت‌کننده تعدادی کتاب به‌عنوان جایزه بدهند، اما باید شرایط زیر رعایت شود:

- هر شرکت‌کننده حداقل یک کتاب دریافت کند.
- شرکت‌کنندگانی که امتیاز بیشتری دارند، باید کتاب‌های بیشتری نسبت به همسایه‌های خود دریافت کنند.

با توجه به امتیاز شرکت‌کنندگان، به روش حریصانه، حداقل تعداد کتاب‌هایی که باید توزیع شود را بیابید.

تحلیل و آنالیز مسئله:

ورودی: آرایه‌ای از امتیازات n شرکت‌کننده که در یک صف ایستاده‌اند

شرایط:

هر شرکت‌کننده حداقل یک کتاب دریافت کند

افراد با امتیاز بیشتر باید کتاب‌های بیشتری نسبت به همسایه‌های خود دریافت کنند

خروجی: حداقل تعداد کتاب‌های مورد نیاز

5.2 توضیح الگوریتم: الگوریتم حریصانه در سه مرحله اصلی کار می‌کند:

ابتدا به همه افراد یک کتاب می‌دهیم

از چپ به راست حرکت می‌کنیم:

اگر فرد فعلی امتیاز بیشتری از نفر قبلی دارد، باید کتاب بیشتری بگیرد

از راست به چپ حرکت می‌کنیم:

اگر فرد فعلی امتیاز بیشتری از نفر بعدی دارد، باید کتاب بیشتری بگیرد

5.3 اثبات درستی:

اثبات می‌کنیم که جواب حاصل معتبر است:

شرط اول: هر فرد حداقل یک کتاب دارد (با مقداردهی اولیه)

شرط حداقل یک کتاب:

با مقداردهی اولیه [1,1,1,1,1] این شرط تضمین می‌شود

شرط دوم: هر فرد با امتیاز بیشتر، کتاب بیشتری از همسایه‌هایش دارد (با دو پیمایش)

شرط امتیاز بیشتر = کتاب بیشتر:

پیمایش چپ به راست: تضمین می‌کند هر فرد با امتیاز بیشتر از سمت راست خود، کتاب بیشتری می‌گیرد

پیمایش راست به چپ: تضمین می‌کند هر فرد با امتیاز بیشتر از سمت چپ خود، کتاب بیشتری می‌گیرد

اثبات بهینگی (به روش تناقض):

فرض کنید جواب بهتری وجود دارد

این یعنی می‌توان حداقل یکی از مقادیر را کاهش داد

اما این کار باعث نقض یکی از دو شرط می‌شود:

a) یا فردی کمتر از یک کتاب می‌گیرد
b) یا فردی با امتیاز بیشتر، کتاب کمتری از همسایه‌اش می‌گیرد

5.4 مثال عددی:

فرض کنید آرایه امتیازات به صورت زیر است:

$$scores = [4, 2, 3, 4, 1]$$

گام‌های حل:

1. مقداردهی اولیه:

$$books = [1, 1, 1, 1, 1]$$

هر نفر حداقل یک کتاب

2. پیمایش از چپ به راست:

مقایسه اندیس 0 و 1:

scores [0] = 4 > scores [1] = 2

books [0] = max (books [0], books [1] + 1) = 2

[2, 1, 1, 1, 1]

مقایسه اندیس 1 و 2:

scores [1] = 2 < scores [2] = 3

books [2] = max (books [2], books [1] + 1) = 2

[2, 1, 2, 1, 1]

مقایسه اندیس 2 و 3:

scores [2] = 3 < scores [3] = 4

books [3] = max (books [3], books [2] + 1) = 3

[2, 1, 2, 3, 1]

مقایسه اندیس 3 و 4:

scores [3] = 4 > scores [4] = 1

books [3] = max (books [3], books [4] + 1) = 3

[2, 1, 2, 3, 1]

3. پیمایش از راست به چپ:

مقایسه اندیس 4 و 3:

scores [4] = 1 < scores [3] = 4

حفظ مقادیر فعلی

[2, 1, 2, 3, 1]

مقایسه اندیس 3 و 2:

Scores [3] = 4 > scores [2] = 3

حفظ مقادیر فعلی

[2, 1, 2, 3, 1]

مقایسه اندیس 2 و 1:

scores [2] = 3 > scores [1] = 2

حفظ مقادیر فعلی

[2, 1, 2, 3, 1]

مقایسه اندیس 1 و 0:

scores [1] = 2 < scores [0] = 4

حفظ مقادیر فعلی

[2, 1, 2, 3, 1]

نتیجه نهایی:

- توزیع نهایی کتاب‌ها: [2, 1, 2, 3, 1]
- مجموع کتاب‌های مورد نیاز: 9

5.5 تحلیل پیچیدگی:

زمانی:

مقداردهی اولیه: O(n)

پیمایش چپ به راست: O(n)

پیمایش راست به چپ: O(n)

پیچیدگی کل: O(n)

فضایی:

نیاز به یک آرایه اضافی برای نگهداری تعداد کتاب‌ها: O(n)

این الگوریتم با پیچیدگی خطی و با دو پیمایش ساده، جواب بهینه را تضمین می‌کند.

کد الگوریتم:

کد این الگوریتم حریصانه در زبان برنامه نویسی پایتون نوشته شده است و خروجی مربوط به آن را میتوانید در صفحه بعد مشاهده کنید.

```python
def min_books_needed(scores):
    n = len(scores)
    # مقداردهی اولیه آرایه کتاب‌ها
    books = [1] * n  # هر نفر حداقل یک کتاب

    # بررسی از چپ به راست
    for i in range(1, n):
        if scores[i] > scores[i-1]:
            books[i] = books[i-1] + 1

    # بررسی از راست به چپ
    for i in range(n-2, -1, -1):
        if scores[i] > scores[i+1]:
            books[i] = max(books[i], books[i+1] + 1)

    return sum(books)  # مجموع کل کتاب‌های مورد نیاز
scores = [4, 2, 3, 4, 1]
print(min_books_needed(scores))
```

```
Connected to Python 3.12.7
def min_books_needed(scores):
9
```

شکل 5- کد الگوریتم حریصانه برای اختصاص کتاب

سوال 6 – حداقل زمان برای اتمام کارها

6.1 تحلیل مسئله

سوال 6: حداقل زمان برای اتمام کارها

یک آرایه شامل چندین کار با زمان‌های موردنیاز مختلف به شما داده شده است. K فرد نیروی کار مشابه هم در اختیار داریم. همچنین مدت‌زمانی که این افراد برای انجام یک واحد کار نیاز دارد، مشخص است.

وظیفه‌ی شما این است که با در نظر گرفتن شرایط زیر، بررسی کنید که آیا می‌توان تمام کارها را در زمان T انجام داد یا خیر:

- هر نیروی کار فقط می‌تواند وظایف متوالی انجام دهد. برای مثال، اگر به یک نیروی کار کارهای ۱ و ۲ اختصاص داده شود، دیگر نمی‌توان کار ۳ را به او نداد و سپس کار ۴ را به او سپرد.
- یک کار نمی‌تواند بین دو نیروی کار تقسیم شود.

در این مسئله:

یک آرایه از چندین کار با زمان‌های موردنیاز مختلف داریم

k نفر نیروی کار مشابه در اختیار داریم

مدت زمان انجام هر واحد کار توسط هر نیرو مشخص است

باید بررسی کنیم آیا می‌توان تمام کارها را در زمان T انجام داد یا خیر

محدودیت‌های مسئله:

1. هر نیروی کار فقط می‌تواند وظایف متوالی انجام دهد (نمی‌توان کارها را به صورت غیرمتوالی به یک نیرو اختصاص داد)
2. یک کار نمی‌تواند بین دو نیروی کار تقسیم شود

برای حل این مسئله، می‌توانیم از یک الگوریتم حریصانه استفاده کنیم. ایده اصلی این است که:

1. کارها را به ترتیب نزولی زمان موردنیاز مرتب کنیم
2. هر کار را به نیرویی اختصاص دهیم که زودتر از بقیه آزاد می‌شود

این استراتژی به این دلیل حریصانه است که ابتدا کارهای زمان‌بر را تخصیص می‌دهیم تا از ایجاد بن‌بست در انتهای زمان‌بندی جلوگیری کنیم.

6.2 الگوریتم پیشنهادی

تابع آیا_قابل_انجام_است(کارها[], K, T):

1. کارها را به ترتیب نزولی مرتب کن
2. اگر بزرگترین کار زمانی بیشتر از T نیاز دارد، برگردان خیر
3. ایجاد آرایه زمان_اتمام[K] که همه عناصر آن صفر هستند
4. برای هر کار i در کارها:
   a. پیدا کردن نیروی کاری j که کمترین زمان_اتمام را دارد
   b. اگر زمان_اتمام[j] + زمان کار i > T است، برگردان خیر
   c. در غیر این صورت، زمان_اتمام[j] += زمان کار i
5. برگردان بله (همه کارها قابل انجام هستند)

تحلیل مرحله به مرحله

مرتب‌سازی کارها: ابتدا کارها را به ترتیب نزولی زمان مرتب می‌کنیم. این کار با پیچیدگی زمانی O(n log n) انجام می‌شود (که n تعداد کارها است).

بررسی امکان‌پذیری اولیه: اگر زمان طولانی‌ترین کار از T بیشتر باشد، قطعاً نمی‌توان کارها را در زمان T انجام داد.

تخصیص کارها: برای هر کار، نیرویی را انتخاب می‌کنیم که زودتر از بقیه آزاد می‌شود و کار را به او می‌دهیم.

بررسی محدودیت زمانی: در هر تخصیص، بررسی می‌کنیم که آیا زمان اتمام کار از T تجاوز می‌کند یا خیر.

6.3 مثال عددی

فرض کنید:

- تعداد کارها = 6
- زمان کارها = [5, 4, 3, 6, 2, 8] (ساعت)
- تعداد نیروی کار 3 = (K)
- محدودیت زمانی 10 = (T) ساعت

مراحل اجرای الگوریتم:

1. **مرتب‌سازی کارها**: [8, 6, 5, 4, 3, 2]
2. **بررسی امکان‌پذیری اولیه**: بزرگترین کار 8 ساعت زمان نیاز دارد که از T=10 کمتر است، پس ادامه می‌دهیم.

تخصیص کارها:

کار 1 (8 ساعت): به نیروی کار 1 تخصیص می‌دهیم. زمان‌های اتمام: [8, 0, 0]

کار 2 (6 ساعت): به نیروی کار 2 تخصیص می‌دهیم. زمان‌های اتمام: [8, 6, 0]

کار 3 (5 ساعت): به نیروی کار 3 تخصیص می‌دهیم. زمان‌های اتمام: [8, 6, 5]

کار 4 (4 ساعت): به نیروی کار 3 تخصیص می‌دهیم. زمان‌های اتمام: [8, 6, 9]

کار 5 (3 ساعت): به نیروی کار 2 تخصیص می‌دهیم. زمان‌های اتمام: [8, 9, 9]

کار 6 (2 ساعت): به نیروی کار 1 تخصیص می‌دهیم. زمان‌های اتمام: [10, 9, 9]

نتیجه: همه کارها در زمان T=10 قابل انجام هستند.

6.4 تحلیل زمانی الگوریتم

- مرتب‌سازی کارها: $O(n \log n)$
- یافتن نیروی کار با کمترین زمان اتمام: $O(K)$ در هر تکرار
- کل حلقه تخصیص کارها: $O(n \times K)$

بنابراین پیچیدگی زمانی کل: $O(n \log n + n \times K)$

در بدترین حالت، اگر تعداد نیروهای کار زیاد باشد، پیچیدگی به $O(n \times K)$ نزدیک می‌شود. می‌توانیم با استفاده از صف اولویت (min-heap) برای نگهداری زمان‌های اتمام، پیچیدگی را به $O(n \log K)$ کاهش دهیم.

6.5 اثبات درستی الگوریتم

الگوریتم حریصانه ما مبتنی بر این ایده است که کارهای بزرگتر را زودتر تخصیص دهیم. این رویکرد برای این مسئله بهینه است زیرا:

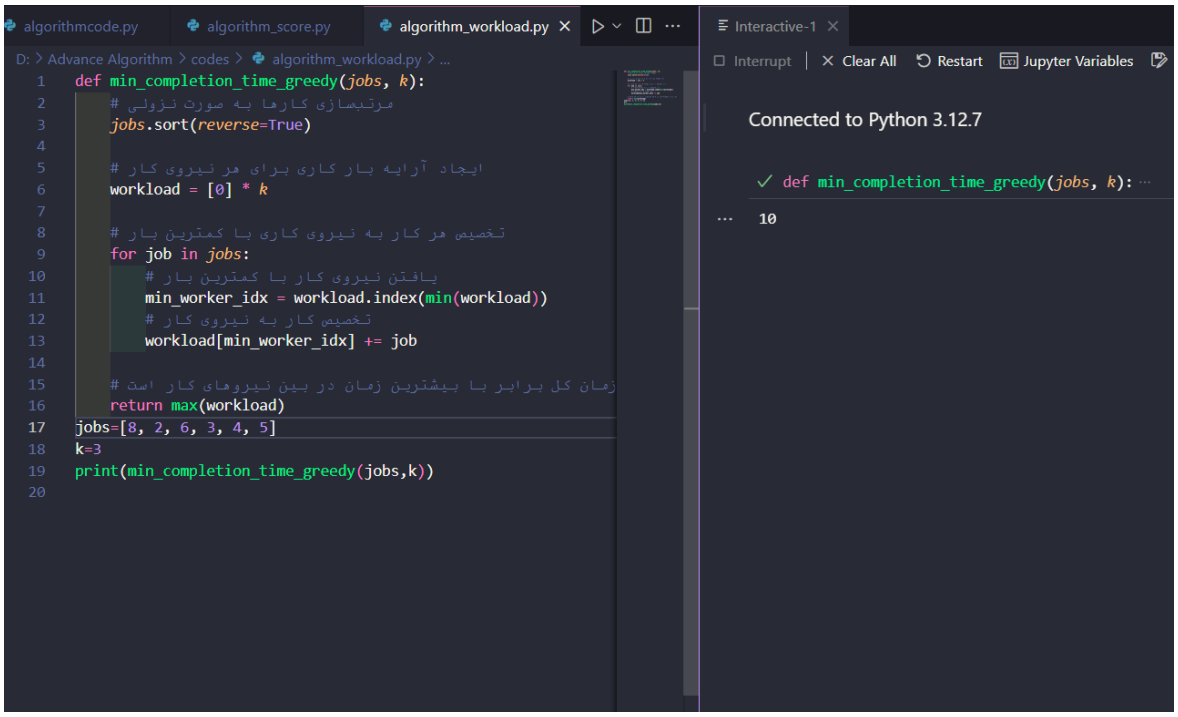
1. اگر نتوانیم کارهای بزرگ را در ابتدا تخصیص دهیم، قطعاً در انتهای زمان‌بندی با مشکل مواجه خواهیم شد.
2. با تخصیص کارهای کوچکتر در انتها، انعطاف‌پذیری بیشتری برای پر کردن فضاهای خالی در زمان‌بندی داریم.

تحلیل محدودیت‌های مسئله

در پیاده‌سازی الگوریتم حریصانه، تمام محدودیت‌های مسئله را در نظر گرفته‌ایم:

1. **انجام متوالی کارها توسط هر نیروی کار:**
   - در الگوریتم، هر نیروی کار کارها را به صورت متوالی انجام می‌دهد و زمان انجام هر کار به زمان قبلی اضافه می‌شود.
2. **عدم تقسیم کارها بین نیروهای کار:**
   - هر کار به طور کامل به یک نیروی کار اختصاص داده می‌شود و تقسیم نمی‌شود.
3. **تعداد محدود نیروی کار (K نفر):**
   - دقیقاً K نیروی کار در آرایه workload در نظر گرفته شده است.

در ادامه کد این الگوریتم هم در زبان برنامه نویسی پایتون قرار داده شده است تا مقدار max workload را محاسبه کند از طریق این کد و با بررسی T می‌توانیم متوجه شویم که آیا این کارها با این تعداد نیروی کار در این زمان T قابل انجام هستند یا خیر.



شکل 6- کد الگوریتم سوال 6 مربوط به **workload**

((پایان))